Scientific Computing WS25/26

Negar Rezaei Nejad

L03 - Topic 2

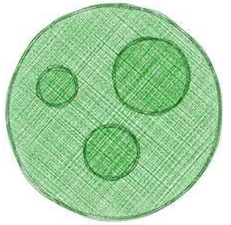
(Rust) Understanding Ownership: What is Ownership?

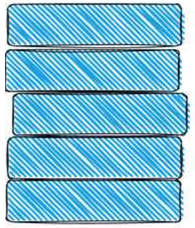
All programs have to manage the way they use a computer's memory while running.

Ownership is a set of rules that govern how a Rust program manages memory.

Memory Management Approach	How It Works	Runtime Cost?	Language Example
Garbage Collection (GC)	A runtime process regularly finds and frees unused memory	Yes	Python, Java, Go
Manual Management	A programmer must explicitly allocate and free memory blocks	No	C, C++
Ownership System	The compiler checks a set of rules about how memory is used; code won't compile if rules are violated	No	**Rust**

Stack and Heap





Both the stack and the heap are parts of memory available to your code to use at runtime.

Importance? Affects how the language behaves and why you have to make certain decisions.

Stack

Stores values in the order it gets them and removes the values in the opposite order. **Last in, first out**.

- Adding data is pushing; removing data is popping.
- All data stored on the stack must have a known, fixed size.
- When a function is called, its passed values and local variables get pushed onto the stack, and when the function is over, they get popped off the stack.

Heap

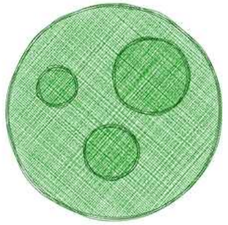
Less organized and is used for data with an unknown or changing size at compile time.

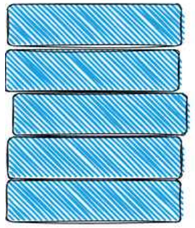
- **Allocating on the heap:** Memory allocator finds a big enough spot for data, marks it as used, and returns a pointer (address of that location)

- The pointer to the heap is a known, fixed size → Can be stored on the **stack**

- When wanting the actual data → Must follow the **pointer**

you must follow the pointer





Stack and Heap

Both the stack and the heap are parts of memory available to your code to use at runtime

Importance? Affects how the language behaves and why you have to make certain decisions

- **Allocating on the heap** is **slower** than **pushing to the stack** because the allocator must search for space and perform bookkeeping.
- **Accessing data in the heap** is generally **slower** because you have to follow a pointer, and the data can be far apart in memory.

The main purpose of ownership

Managing **heap** data

including keeping track of what code is using what data, minimizing duplication, and cleaning up unused data

Ownership Rules:

- Each value in Rust has an owner.
- There can only be one owner at a time.
- When the owner goes out of scope, the value will be dropped.

Variable Scope

A scope is the range within a program for which an item is valid.

```
fn main() {
    {                      // s is not valid here, since it's not yet declared
        let s = "hello";   // s is valid from this point forward

        // do stuff with s
    }                      // this scope is now over, and s is no longer valid
}
```

The String Type: With Ownership Focus

The String type is the core example of Ownership because it manages **dynamic**, **variable-sized data** on the **Heap**. These aspects also apply to other complex data types.

Feature	Mutability	Memory	Use Case	What it Represents
String	Yes (Can be changed)	Heap (Dynamic Size)	User input, text manipulation	Text that is dynamic, built at runtime.
String Literal	No (Immutable)	Stack (Fixed Size)	Hardcoded, static values	Text that is hardcoded and known at compile time.

The String Type: With Ownership Focus

You can create a String from a string literal using the *from* function.

```rust
fn main() {
    let mut s = String::from("hello");

    s.push_str(", world!"); // push_str() appends a literal to a String

    println!("{s}"); // this will print `hello, world!`
}
```

Why can String be mutated but literals cannot?

The difference is in how these two types deal with **memory**.

Memory and Allocation

Handling dynamic data like String requires two steps:

- Requesting memory (**allocation**)

  Done by us: when we call String::from

- Releasing it (**freeing**)

  When a variable goes out of scope, Rust automatically calls a special function called *drop* at the closing curly bracket, and the memory is automatically returned.

Variables and Data Interacting with Move

```rust
fn main() {
    let x = 5;
    let y = x;
}
```

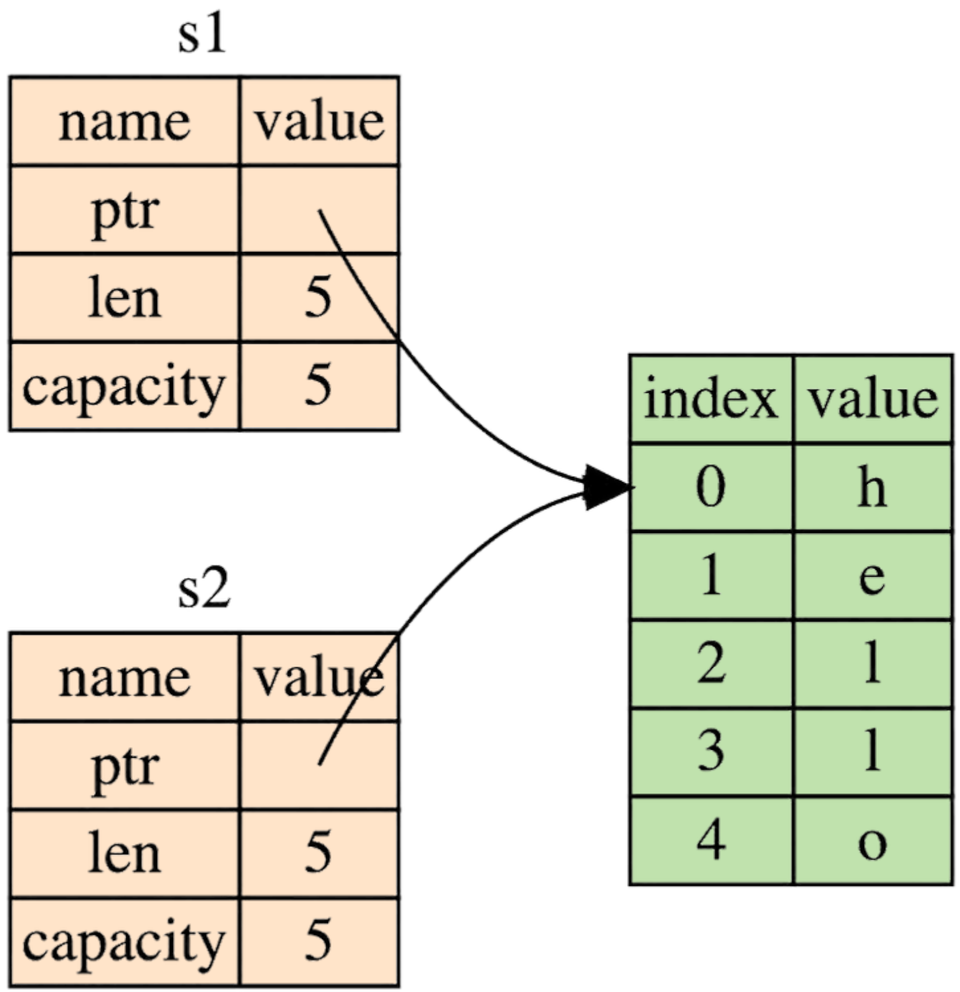
Bind the value 5 to x; then make a copy of the value in x and bind it to y.

We now have two variables, x and y, and both equal 5. Integers are simple values with a known, fixed size, and these two 5 values are pushed onto the **stack**.

```rust
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;
}
```

Same?

A pointer to the stored on the stack memory that holds the contents of the string, a length, and a capacity.



The memory on the heap that holds the contents

Problem?

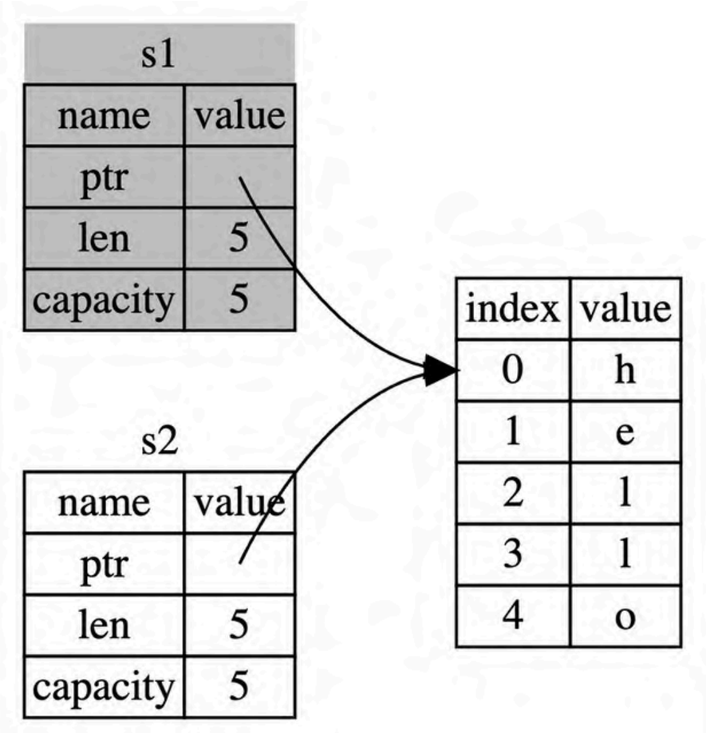
when s2 and s1 go out of scope, they will both try to free the same memory. This is known as a **double free error**.

To ensure memory safety, after the line let s2 = s1;, Rust considers s1 as **no longer valid**.

```rust
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;

    println!("{s1}, world!");
}
```

In this example, we would say that s1 was **MOVE**d into s2, because Rust also invalidates the first variable.

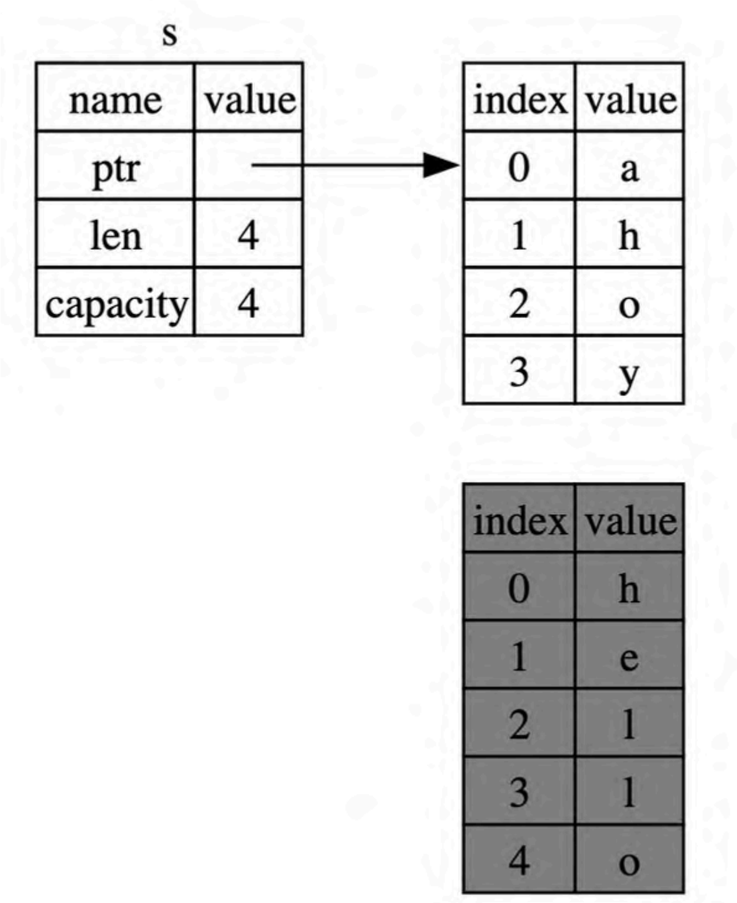


Scope and Assignment

```
let mut s = String::from("hello");
s = String::from("ahoy");

println!("{s}, world!");
```

When you assign a completely **new value to an existing variable**, the original string thus immediately goes out of scope. Rust will call drop and free the original value's memory immediately.



Variables and Data Interacting with Clone

```rust
fn main() {
    let s1 = String::from("hello");
    let s2 = s1.clone();

    println!("s1 = {s1}, s2 = {s2}");
}
```

If we **do** want to deeply copy the heap data of the String, not just the stack data, we can use a common method called *clone*.

```rust
fn main() {
    let x = 5;
    let y = x;

    println!("x = {x}, y = {y}");
}
```

```
x = 5, y = 5
```

we don’t have a call to clone, but x is still valid and wasn’t moved into y. **Why?**

Stack-Only Data: Copy

The Copy trait is applied to **fixed-size types** stored only on the Stack, avoiding the Ownership move rule and keeping the original variable valid. Calling clone wouldn't do anything different from the usual shallow copying.

Some of the types that implement Copy:

- All the integer types, such as **u32**.
- The **Boolean** type, bool, with values true and false.
- All the floating-point types, such as **f64**.
- The character type, **char**.
- **Tuples**, if they only contain types that also implement Copy.

  For example, (i32, i32) implements Copy, but (i32, String) does not.

Ownership and Functions

Passing a variable to a function will **move** or **copy**, just as assignment does.

```rust
fn main() {
    let s = String::from("hello");  // s comes into scope

    takes_ownership(s);             // s's value moves into the function...
                                    // ... and so is no longer valid here

    let x = 5;                      // x comes into scope

    makes_copy(x);                  // Because i32 implements the Copy trait,
                                    // x does NOT move into the function,
                                    // so it's okay to use x afterward.

} // Here, x goes out of scope, then s. However, because s's value was moved,
  // nothing special happens.

fn takes_ownership(some_string: String) { // some_string comes into scope
    println!("{some_string}");
} // Here, some_string goes out of scope and `drop` is called. The backing
  // memory is freed.

fn makes_copy(some_integer: i32) { // some_integer comes into scope
    println!("{some_integer}");
} // Here, some_integer goes out of scope. Nothing special happens.
```

Return Values and Scope

Returning a value from a function also transfers its ownership.

When a complex type like *String* is returned, its ownership moves out of the function and into the variable that receives it. To let a function use a value without taking permanent ownership, you must move the original value back out as a return value.

```
fn main() {
    let s1 = String::from("hello");

    let (s2, len) = calculate_length(s1);

    println!("The length of '{s2}' is {len}.");
}

fn calculate_length(s: String) -> (String, usize) {
    let length = s.len(); // len() returns the length of a String

    (s, length)
}
```

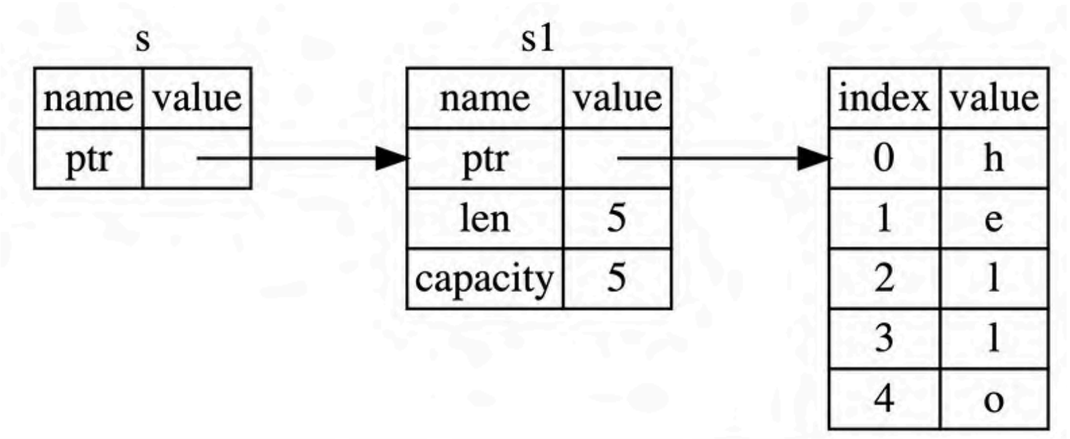
```
The length of 'hello' is 5.
```

(Rust) Understanding Ownership: References and Borrowing

To solve the problem of returning ownership, Rust uses references, which are like **a pointer in that it's an address we can follow to access the data stored at that address** without taking ownership. This action is called borrowing.

When functions have references as parameters instead of the actual values, we won't need to return the values in order to give back ownership, because we never had ownership.

- **Syntax**: References are created using the ampersand (&) operator.
- **No Cleanup:** Because a reference does not own the data it points to, the data is not dropped when the reference goes out of scope, allowing the original owner (s1) to remain valid.




```rust
fn main() {
    let s1 = String::from("hello");

    let len = calculate_length(&s1);

    println!("The length of '{s1}' is {len}.");
}

fn calculate_length(s: &String) -> usize { // s is a reference to a String
    s.len()
} // Here, s goes out of scope. But because s does not have ownership of what
  // it refers to, the String is not dropped.
```

```
The length of 'hello' is 5.
```

- **Immutability:** References are immutable by default. You cannot modify data through a regular & reference, as attempting to do so results in a compile-time error.

```rust
fn main() {
    let s = String::from("hello");

    change(&s);
}

fn change(some_string: &String) {
    some_string.push_str(", world");
}
```

Key Idea

Borrowing allows a function to use a value without having to move ownership back and forth, significantly simplifying code.

Mutable References

To modify a borrowed value with just a few small tweaks that use, instead, a mutable reference.

```rust
fn main() {
    let mut s = String::from("hello");

    change(&mut s);
}

fn change(some_string: &mut String) {
    some_string.push_str(", world");
}
```

```rust
let mut s = String::from("hello");

let r1 = &mut s;
let r2 = &mut s;

println!("{r1}, {r2}");
```

One big restriction

If you have a mutable reference to a value, you can have no other references to that value.
r1 **must last** until it's used in the println!.

The point of restriction?

The benefit of having this restriction is that Rust can prevent *data races* at compile time.

Happens when these three behaviors occur:

- Two or more pointers access the same data at the same time.
- At least one of the pointers is being used to write to the data.
- There's no mechanism being used to synchronize access to the data.

Rust prevents this problem by refusing to compile code with data races!

Rust enforces a similar rule for combining mutable and immutable references.

```rust
fn main() {
    let mut s = String::from("hello");

    let r1 = &s; // no problem
    let r2 = &s; // no problem
    let r3 = &mut s; // BIG PROBLEM

    println!("{r1}, {r2}, and {r3}");
}
```

NOTE!

Multiple immutable references are allowed because no one who is just reading the data has the ability to affect anyone else's reading of the data.

A reference's scope starts from where it is introduced and continues through the last time that it is used.
This code **will compile** because the last usage of the immutable references is in the println!, before the mutable reference is introduced.

```rust
fn main() {
    let mut s = String::from("hello");

    let r1 = &s; // no problem
    let r2 = &s; // no problem
    println!("{r1} and {r2}");
    // Variables r1 and r2 will not be used after this point.

    let r3 = &mut s; // no problem
    println!("{r3}");
}
```

Dangling References

A dangling pointer:

a pointer that references a location in memory that may have been given to someone else—by freeing some memory while preserving a pointer to that memory.

```
fn dangle() -> &String { // dangle returns a reference to a String

    let s = String::from("hello"); // s is a new String

    &s // we return a reference to the String, s
} // Here, s goes out of scope and is dropped, so its memory goes away.
  // Danger!
```

Rust won’t let us do this.

The compiler will ensure that the data will not go out of scope before the reference to the data does.

Programming problem

Write a function that takes a string of words separated by spaces and returns the first word it finds in that string. If the function doesn't find a space in the string, the whole string must be one word, so the entire string should be returned.

An Attempt

```rust
fn first_word(s: &String) -> usize {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return i;
        }
    }

    s.len()
}
```

takes a string of words separated by spaces and returns the index of the end of the first word in the string

A Problem: IT RUNS!

```rust
fn first_word(s: &String) -> usize {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return i;
        }
    }

    s.len()
}

fn main() {
    let mut s = String::from("hello world");

    let word = first_word(&s); // word will get the value 5

    s.clear(); // this empties the String, making it equal to ""

    // word still has the value 5 here, but s no longer has any content that we
    // could meaningfully use with the value 5, so word is now totally invalid!
}
```

```
No output
```

We could use the value 5 with the variable s to try to extract the first word out, but this would be a bug.

Because when the string s is later cleared (s.clear()), the original string is emptied, but the number 5 still exists in the variable word.

String Slices

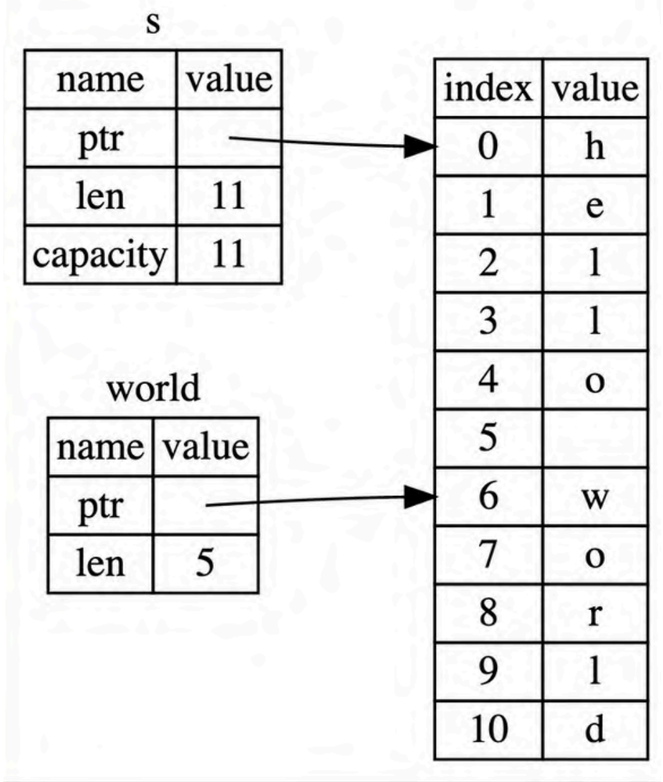
Comes to the rescue!

A string slice is a reference to a contiguous sequence of the elements of a String

```rust
fn main() {
    let s = String::from("hello world");

    let hello = &s[0..5];
    let world = &s[6..11];
}
```

In the case of let world = &s[6..11];, world would be a slice that contains a pointer to the byte at **index 6** of s with a **length** value of **5**.



Programming problem

Write a function that takes a string of words separated by spaces and returns the first word it finds in that string. If the function doesn't find a space in the string, the whole string must be one word, so the entire string should be returned.

```rust
fn first_word(s: &String) -> &str {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return &s[0..i];
        }
    }

    &s[..]
}
```

When we find a space, we return a string slice using the start of the string and the index of the space as the starting and ending indices.

We get back a single value that is **tied** to the underlying data.

Compiling this code will produce an error.

By using a slice (&str), you tie the validity of the first word to the validity of the original string, and the compiler prevents any action that could break that link.

```rust
fn first_word(s: &String) -> &str {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return &s[0..i];
        }
    }

    &s[..]
}

fn main() {
    let mut s = String::from("hello world");

    let word = first_word(&s);

    s.clear(); // error!

    println!("the first word is: {word}");
}
```

Other Slices

Just as we might want to refer to part of a string, we might want to refer to part of an array.

```
let a = [1, 2, 3, 4, 5];

let slice = &a[1..3];

assert_eq!(slice, &[2, 3]);
```

(Rust) An I/O Project: Building a Command Line Program

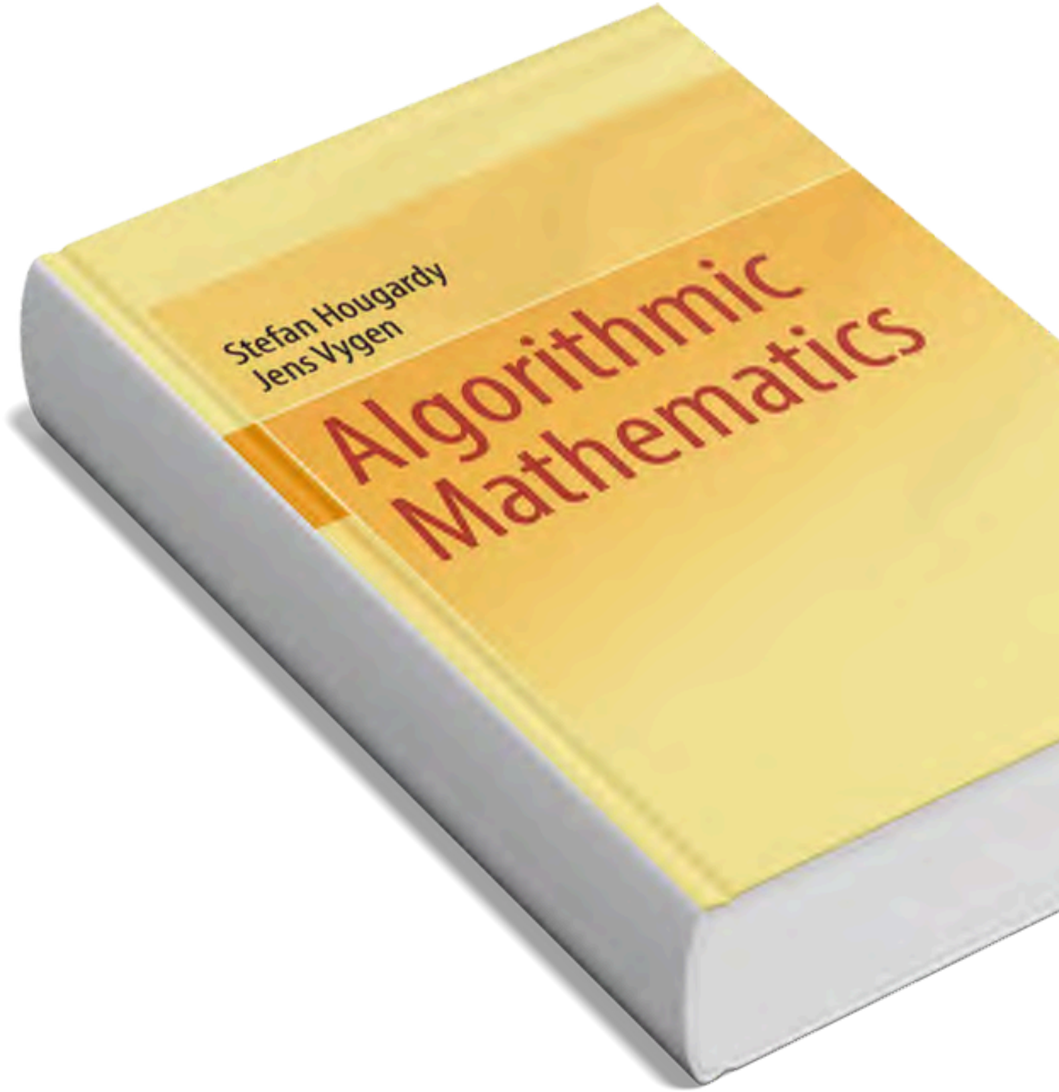
The chapter walks through building a practical command-line application called ***minigrep***.
The entire project is designed to teach how to organize and handle **I/O (Input/Output)** in a Rust program.

Focus:

- **Handling Arguments:** Learn to accept user input (query and file path) directly from the command line.
- **Structuring Data:** Define a *Config* struct for clean, organized handling of program arguments.
- **Error Handling:** Implement graceful error management using Result, avoiding application crashes.
- **Modular Design:** Separate the program logic into a Binary Crate and a Library Crate for better testing.
- **Core I/O Logic:** Implement File I/O (reading contents) and the core search algorithm.
- **Enhancements:** Add features like case-insensitive search using environment variables.

The book's adjacent chapters deal with complex math like arbitrarily large integers and Euclidean algorithms.

These concepts highlight the need for **dynamic memory management** (the Heap) and precise resource control—the exact, difficult problem that Rust's Ownership system solves elegantly and automatically.



Your next step is to master these fundamentals. :)

THANK YOU!